Affine proof cores and incremental machine transport

Further proof simplification research

21 October 2026

Abstract

The leading new reusable proof interface contracts 1,059 recognized drain loops in K215's canonical source graph into total affine updates. The remaining auxiliary proof graph has 367 vertices, 732 edges and only 33 shared matrices. Separately, context215 can inherit the certified K215 endpoint through six short whole-tape blocks and nine histories from the old table, rather than rebuilding its site and startup proofs. New logical patches remove a third unnecessary formula induction and narrow the model interface. These results are independent executable checks and uncompiled generic proof proposals, not a new kernel build, state reduction or measured speedup. Recent archives already contain compiled normalization and zero-offset source proofs; the report distinguishes those records from maintained adoption and from the new work.

1 What the recent commits already establish

The final reviewed main commit is

`62f0f95e2b2df5b380ae502eb65bd64f1ee90209`

The initial review at `53269bc` found three commits beyond the prior pin: batch 204 places six 215-state alternatives; a subsequent commit adds six proof proposal archives; the latest adds another proof-simplification archive. Five subsequent commits place batches 205–207, including previously inspected archives and a finite-replay/uncompiled-draft packet. The entire maintained Lean and Coq Git subtrees are unchanged across both refreshes. The previously adopted symmetry theorem remains maintained; the newly delivered archives are not automatically production adoption.

Two delivered records advance the frontier. One reports a compiled 1,426-entry normalized source checker, 2,851 branch checks including initialization, 22 new local modules and the exact all-register source-halting equivalence. Independent inspection matches all 22 source hashes and zero-exit compilation records. Its endpoint assumptions are `propext`, `Classical.choice` and `Quot.sound`; its generic checker uses `propext` and `Quot.sound`. It already combines pointer normalization with ordinary, zero-offset bisimulation, although it still stores paired correspondence.

The latest second archive records a 70-module ordinary zero-offset source lane. All included source and raw-log hashes match its ledger, and the offset-bisimulation module is absent from its import closure. Its generic checker is 113 lines; source data and facade total 224,084 bytes versus 456,548 upstream. Its input-dependent pointer fuel avoids the earlier 128-bit truncation trap. These are recorded isolated/incremental builds and source-size measurements, not a local compilation, proof-object comparison, or rebuilt full binary-to-ZFC endpoint.

Thus another normalized 1,426-entry checker or the already reported 1,423-class labelled quotient is not the new contribution. The opportunities below go beyond them. Archive hashes, receipts and the two relevant archives are included as inert evidence; no received executable was run.

2 Contract 1059 drain loops into a 367 vertex core

For the currently identical canonical graphs, direct normalized or canonical equality remains the simpler first choice. Drain contraction introduces extra shape, rank and arithmetic checks and may increase verification work. Its strongest use is a reusable macro interface for future sources with different transfer/clear-loop implementations, not a promised faster proof of the current identical pair.

The two current source tree literals remain exactly equal to the earlier inspected literals. Regenerating their canonical increment-weighted test graph gives the same 1,426 vertices. Of these, 1,059 have a particularly simple total-loop shape.

At vertex v, test register r. Suppose its nonzero edge returns to v and adds a fixed vector c, where $c_r = 0$; its zero edge exits to $w \neq v$ and adds d. The graph's nonzero transition decrements r *before* adding c. Starting from an arbitrary natural register file R, set $n = R_r$. Exactly n nonzero iterations drain r to zero, then one zero transition exits. The final update is

$$F(R)_r = d_r, \qquad F(R)_j = R_j + R_r c_j + d_j \quad (j \neq r).$$

One induction on n proves this formula and the exact $n + 1$ graph steps. No bound on register values or invariant about blank startup is required. The exit constants are added once, including a possibly nonzero d_r.

Represent F by a 10×10 homogeneous matrix on $(R_0, \ldots, R_8, 1)^T$. The tested row contains only its exit constant; every other row retains its identity term, the appropriate coefficient in column r, and its exit constant. All coefficients are nonnegative integers and the last row preserves one.

2.1 Composition and progress

Retain non-drain tests and halt. From each retained branch, follow its increment update and every subsequent drain exit until the next retained vertex, composing the matrices in execution order. The retained nonzero branch's decrement remains explicit *before* the matrix. It cannot be silently absorbed into an unsigned affine matrix.

Every eliminated vertex has a checked natural rank; a drain-to-drain exit strictly decreases it. Maximum rank is six. This excludes infinite chains of individually terminating macros from being hidden inside a contracted edge. All 1,059 removed vertices occur in checked core-edge paths. Halt is retained and has no outgoing edge. The initial vertex is already retained, so its entry matrix is identity.

The resulting data are:

Retained vertices	367 (366 tests and one halt)
Labelled edges	732
Distinct local drain shapes	24
Distinct shared affine matrices	33
Largest matrix coefficient	4
Longest contracted drain chain	6

A generic contraction theorem expands each core edge into finite original runs and contracts original runs back, using totality and strict rank decrease. Each core transition retains its own positive-duration decrement-test step. The theorem preserves halting, and can preserve the final register file. Composing it with the source-to-canonical-graph adequacy theorem preserves the original endpoint.

2.2 A small trusted checker rather than a trusted optimizer

A proof-assistant checker should verify the drain shape and $c_r = 0$, rank decreases, literal path successors, ordered matrix products, entry data, branch registers and halt markers. The optimizer and shared dictionary remain untrusted. Compare all coefficients, not hashes. One generic arithmetic induction covers all 1,059 instances; there is no need for 1,059 separate loop inductions.

The owned checker verifies matrix products independently by multiplication and symbolic coordinate updates, checks rank/coverage obligations and compact expansion, and runs 5,295 concrete local regressions. A separate validator checks 1,098 complete core-edge runs, including the retained decrement, and rejects nine adversarial mutations, including a self-incrementing drain, nondecreasing rank, missing core decrement, wrong entry transform, reversed matrix composition and malformed increment/register data. The regressions are not the all-natural proof; the induction above is the proposed universal argument. Source-to-graph adequacy and the new contraction checker still need proof-assistant implementation and compilation.

Ordinary labelled minimization removes only three vertices, and conservative arbitrary-initial-register interval analysis forces only two tests to zero. These checked negative results are scoped to their respective label and interval domains, not lower bounds on stronger semantic minimization. They support prioritizing the drain theorem over more graph partitioning. The 367 vertices are an *auxiliary proof core*, not 367 machine states or a smaller Turing machine. K215 still has 215 working states; no fraction of compilation time has been shown to disappear.

3 Transport context215 using six blocks

The committed context-return table differs from certified native K215 in exactly five rows, with unchanged source. A new independent checker verifies six pairs of short blocks using 28 literal transitions in total. Every read stays in an explicit finite window; each pair ends at the same state and head with every window cell equal. Arbitrary exterior tape is untouched. Both durations are positive, and no strict interior halts or uses a changed row.

The useful specialization is that only histories in the *old* machine need checking. There are five two-edge histories entering state 13 and four immediate incoming histories for states 27, 83, 156 and 198: nine paths in all. One path's latest write contradicts the currently read symbol and is discarded, leaving eight compatible cases. Every compatible history implies a block guard. Start state zero cannot reach a changed entry too early for the history argument, regardless of the initial tape.

At a synchronized full configuration, take a common step if its row is unchanged. Otherwise select a block using the old machine's reachable-history fact, execute the two certified positive blocks, and synchronize their identical endpoints. The next boundary is again reachable in the old machine. No new-machine history invariant is necessary. Positivity and absence of interior halts yield both finite-halting implications by induction on remaining transition count. Consequently, for every initial tape τ and head h,

$$\mathrm{Halts}(K215, 0, h, \tau) \quad \longleftrightarrow \quad \mathrm{Halts}(\mathrm{context215}, 0, h, \tau).$$

This is positive stuttering equivalence, not step-for-step equality. Blank-tape equivalence follows without a separate initializer proof. Once the generic theorem and finite checks are kernel-certified and attached to the exact formal table, the existing source calibration, 6,176 site proofs and startup proof can be reused. The original certified endpoint is still required: this is a much smaller *incremental* proof, not a standalone replacement for its foundations.

Generic old-run positive stuttering, arbitrary-tape state-zero equivalence and endpoint reuse are already

supplied in incoming FM215/M124 transports. The present contribution is an independently checked specialization and obligation count for the exact context215 table, not invention of that method. No exact compiled six-block context215 attachment was found in the seven inspected archives; this bounded review is not a global novelty claim.

Recommendation: CM215 has the least remaining Lean integration work. The second simplification archive records successful compilation of `CM215Core.blank_halts_iff_primary` in the repository's actual `KDenseBridge.BlankHalts` semantics, with the exact table compatibility and Core/native bridge. Its reported assumptions are `propext` and `Quot.sound`. The supplied ZFC adapter composes the maintained K215 endpoint, but that final adapter and its full dependency closure were explicitly excluded from the fresh compilation. Compiling that attachment is the next concrete milestone, not an already completed full endpoint.

FM215/M124 have smaller local arguments: two incoming-marker facts and two three-step blocks. However, FM215's recorded all-tape theorem uses standalone function-tape semantics, and M124's recorded Lean/Rocq proofs leave the upstream endpoint-model adapter unintegrated. They first need exact table and tape-semantics bridges. Fewer local clauses do not imply less remaining integration work. Context215's six-block checker is a useful larger instance, but is not the easiest overall candidate to attach. Its exact kernel attachment remains uncompiled here.

3.1 What the other variants do not simplify

Context215 and the shallow-source variant have the same internal decoder interface as certified K215: 281 templates, 234 product states, 468 edges and 97 exposed leaf contexts. Reducing source depth by one does not reduce this proof interface. The shallow source also needs its own reassociation transport.

DR215 has 236 product states and 472 edges versus its ST215 parent's 235 and 470, with 95 leaf contexts in each. Its two changed physical rows affect **139 guarded executions**, not only the four changed source operands or eight branches. All 139 changed executions pass the owned symbolic replay. The other 6,012 of 6,151 executions have disjoint direct evaluator footprints and are candidates for reuse.

That reuse requires a generic dependency theorem covering every dynamic table lookup, branch assumption and box-oracle footprint. The present analyzer assumes the existing box lemmas; it does not reprove them. Disjoint direct row footprints alone do not certify their preservation. These measurements do not establish a smaller standalone physical-runtime proof for DR215.

4 Further remove logical proof dependencies

The previous model patch removed two syntactic round-trip inductions. A third induction, positivity of every encoded formula, is unnecessary for the padded matrix bound. The right projection inequality alone gives

$$\mathrm{enc}(\neg\neg\psi) = 4P(\mathrm{enc}(\neg\psi), 1) \geq 4 > 1.$$

The new proposed patch removes all three inductions and uses the existing semantic encode/decode equivalence where semantics, rather than syntax, is needed. It reduces the reviewed file from 270 to 235 lines: ten more lines than the prior patch. The final seed theorem statement and numerical calculus are unchanged. External callers of deleted public helper lemmas still need checking or temporary compatibility lemmas. The proposal is uncompiled.

The model hypothesis can also be narrowed from all numerically derivable named formulas to the exact seed schema used: equality-free double-negation-padded bundles at roles zero and one. Each has a direct derivability proof, and the already generic model-completeness theorem can be invoked on this

smaller theory. This exposes a cleaner interface without a new model-consistency argument. It does not restrict the raw upper soundness theorem or the machine's enumeration.

A small proposed axiom-transport kernel fixes truth, implication and theorem-only generalization over an arbitrary carrier and varies only an axiom predicate. To transport all derivations, prove that every source axiom is derivable in the target; mutual admissibility gives equality of theorem predicates. No injective syntax encoding or well-formed-code assumption is needed. DR215's alternate sixth-row sampler has the same all-natural axiom image after renaming witnesses, but a fixed AX tuple does not have identical outputs, so its repaired compiler block remains necessary. Antecedent-exchange variants require admissibility in *both* directions from the unchanged common basis, not a circular appeal to the row being replaced.

Other small opportunities include a shared generic propositional projection, proved using the existing classical tautology interface, and removal of irreflexivity from the helper `matrix_iff`, whose proof does not use it. Irreflexivity remains essential for neighboring negation/bundle semantic lemmas. None of these local improvements removes equality interpretation, freshness, quotient construction, or the model-theoretic calibration to ZF/ZFC. A generic axiom would hide those obligations rather than simplify their proof.

5 Reproduction and next milestones

The packet contains the exact graph, rank and matrix certificates, owned graph and runtime checkers, inert pinned sources/tables, local mutation tests, proposed Lean patches, historical build evidence and provenance. Run with Python 3 and assertions enabled, using a fresh external directory:

```
python -I -B verify.py --work /tmp/next-proof-check
```

It regenerates the canonical input graph, checks current tree-literal equality, recomputes contraction and auxiliary analyses, checks all patch and decoder premises, replays the DR dependency census, and checks logical patch pins. It does not compile Lean/Rocq or convert historical recorded builds into a local verification claim.

First compile the supplied CM215 final endpoint adapter and integrate the already recorded normalized source lane into maintained consumers. Compile the smaller logical patch independently. Develop the drain theorem and checker when a genuinely differing loop implementation benefits from the affine interface; reuse the existing stutter theorem rather than rebuilding it. Benchmark only after preserving endpoint statements and axiom reports. No new state reduction, internal-PA theorem, or measured proof-assistant speedup is claimed in this round.